ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

QUẢN TRỊ MẠNG VÀ HỆ THỐNG

Đề tài:

Network Load Balancing – HAProxy

Giảng viên hướng dẫn: Th.S Trần Thị Dung

Th.S Văn Thiên Luân

Sinh viên thực hiện: Nhóm 16

Nguyễn Lê Nhật Đăng - 23520231

Trần Hải Đăng - 23520237

Huỳnh Minh Đạt - 23520249

Thành phố Hồ Chí Minh, ngày 30 tháng 05 năm 2025

MỤC LỤC

PHẦN 1: TỔNG QUAN 3

1.1. Tổng quan về Load Balancing 4

1.1.1 Cách thức hoạt động của Load Balancing 4

1.1.2 Các phương pháp Load Balancing phổ biến 5

1.1.3 Các chế độ hoạt động của Load Balancer 6

1.2. Giới thiệu về HAProxy 8

1.3. Mô hình và cách hoạt động của HAProxy 8

PHẦN 2: TRIỂN KHAI 11

2.1. Mô hình 11

2.2. Các bước cài đặt HAProxy 11

2.3. Cấu hình HAProxy trên Layer 4 12

2.3.1. Thử nghiệm với HAProxy Layer 4 17

2.4. Cấu hình HAProxy trên Layer 7: HTTP 20

2.4.1. Thử nghiệm với HAProxy Layer 7 22

2.5. Trên Layer 7, chỉnh cấu hình HAProxy để có thể kết nối giữa HAProxy và các Client theo chuẩn mã hoá SSL/TLS. 24

2.6. Triển khai HAProxy ACL để phân phối các yêu cầu tới các máy chủ. 31

2.6.1. Access Control List 31

2.6.2. Cú pháp khai triển một Access Control List 32

2.6.3. Áp dụng ACL vào cấu hình HAProxy 32

DANH MỤC BẢNG

Bảng 1: Bảng chi tiết thành phần trong mạng 11
Bảng 2: Bảng phân chia công việc 38
Bảng 3: Bảng tự đánh giá thành viên 39

DANH MỤC HÌNH ẢNH

Hình 1: Kiến trúc cân bằng tải HAProxy – Mô hình logic 9
Hình 2: Mô hình mạng triển khai trong thực tế sử dụng HAProxy 11
Hình 3: Tải Docker Image cho HAProxy từ Docker Hub 12
Hình 4: Giao diện phản hồi Web Server 1 thông qua HAProxy 12
Hình 5: Kết quả Client 1 gửi request tới Server HAProxy 18
Hình 6: Kết quả Client 2 gửi request tới Server HAProxy 18
Hình 7: Kết quả Client 3 gửi request tới Server HAProxy 18
Hình 8: HAProxy phân phối tải đến các Web Server qua log truy cập TCP 19
Hình 9: Mô phỏng sự cố Web Server bằng cách tắt đi Web1 trong Docker Desktop 19
Hình 10: Client 1 được chuyển tiếp tới Web Server 2 sau khi ngừng hoạt động Web Server 1 19
Hình 11: Client 2 được chuyển tiếp tới Web Server 3 sau khi ngừng hoạt động Web Server 1 20
Hình 12: Client 3 được chuyển tiếp tới Web Server 2 sau khi ngừng hoạt động Web Server 1 20
Hình 13: HAProxy phân phối tải tới Web Server 2 và 3 thông qua log truy cập TCP 20
Hình 14: Giao diện các Web Server 22
Hình 15: Client gửi request đến Server HAProxy lần đầu tiên 23
Hình 16: Client gửi request đến Server HAProxy lần thứ hai 23
Hình 17: Client gửi request đến Server HAProxy lần thứ ba 23
Hình 18: Log truy cập HTTP từ Client đến Backend Server – Mã trạng thái 200 24

Hình 19: Log truy cập HTTP thông báo rằng Server bị lỗi với mã trạng thái là 400 24
Hình 20: Mô hình triển khai thực tế trong HAProxy với giao thức SSL/TLS tại Layer 7 25
Hình 21: Cấu hình ánh xạ tên miền cho các Webserver trong file host (Windows) 29
Hình 22: Client gửi yêu cầu đến Server HAProxy, sau đó được chuyển tiếp tới trang uit.edu.vn:443 29
Hình 23: Client gửi yêu cầu đến Server HAProxy, sau đó được chuyển tiếp tới trang google.com:443 30
Hình 24: Client gửi yêu cầu đến Server HAProxy, sau đó được chuyển tiếp tới trang duckduckgo.com:443 30
Hình 25: Đối với một số trang web không yêu cầu xác thực chứng chỉ tự kí - kết nối an toàn 30
Hình 26: Chứng chỉ không được chấp nhận - cần xác thực từ bên CA thứ ba uy tín 31
Hình 27: Log phân tích chỉ ra chứng chỉ không được xác thực 31
Hình 28: Mô hình triển khai thực tế đối với HAProxy - ACL 32
Hình 29: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy1.com:443 lần thứ nhất 35
Hình 30: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy1.com lần thứ hai. 36
Hình 31: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy1.com:443 lần thứ ba 36
Hình 32: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy2.com:443 37
Hình 33: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy3.com:443 37

PHẦN 1: TỔNG QUAN

1.1. Tổng quan về Load Balancing

Load Balancing (cân bằng tải) được định nghĩa là phân phối lưu lượng truy cập mạng hoặc ứng dụng một cách có hiệu quả trên nhiều server trong một cụm server.

Mục tiêu chính là tối ưu hóa việc sử dụng tài nguyên, tối đa hóa thông lượng, giảm thiểu thời gian phản hồi và ngăn chặn tình trạng quá tải xảy ra trên bất kỳ tài nguyên riêng lẻ nào.

Ngoài việc phân tán lưu lượng, Load Balancing còn đóng vai trò quan trọng trong việc cải thiện tính khả dụng của ứng dụng. Bằng cách chia sẻ khối lượng công việc trên nhiều tài nguyên dự phòng, hệ thống có thể tiếp tục hoạt động ngay cả khi một hoặc một số máy chủ gặp sự cố. Khả năng này không chỉ tăng cường độ tin cậy mà còn cho phép thực hiện các hoạt động bảo trì hoặc nâng cấp trên máy chủ mà không gây gián đoạn dịch vụ cho người dùng. Sự phát triển của các ứng dụng Web phức tạp và yêu cầu về trải nghiệm người dùng liên tục đã củng cố vai trò không thể thiếu của Load Balancing trong kiến trúc hệ thống hiện đại.

Sự cần thiết của Load Balancing ngày càng trở nên rõ ràng khi các ứng dụng Web phải đối mặt với hàng triệu yêu cầu đồng thời, đòi hỏi một kiến trúc có khả năng phân tán tải để duy trì hiệu suất và độ ổn định.

1.1.1 Cách thức hoạt động của Load Balancing

Load Balancing là một quá trình phức tạp liên quan đến nhiều bước và thành phần phối hợp để phân phối hiệu quả lưu lượng mạng trên nhiều máy chủ. Một mô hình hoạt động cơ bản của Load Balancing gồm các bước:

- Tiếp nhận yêu cầu từ máy khách: quá trình này bắt đầu khi một máy khách gửi yêu cầu đến một ứng dụng hoặc dịch vụ Web. Thay vì gửi trực tiếp đến máy chủ cụ thể tất cả yêu cầu này sẽ được Load Balancer tiếp nhận.
- Quyết định định tuyến của Load Balancer: Khi Load Balancer nhận được yêu cầu, Load Balancer sẽ dựa trên một số yếu tố như thuật toán cân bằng tải, tình trạng của

của máy chủ, phiên làm việc của người dùng, quy tắc cân bằng tải để quyết định việc chuyển tiếp yêu cầu của máy khách.

- Chuyển tiếp yêu cầu đến máy chủ Backend: Sau khi đã chọn được máy chủ Backend phù hợp, Load Balancer sẽ chuyển tiếp yêu cầu từ máy khách đến máy chủ đó. Quá trình này thường bao gồm việc chỉnh sửa một số thông tin trong gói tin tùy thuộc vào việc cấu hình.
- Xử lý yêu cầu bởi máy chủ Backend: Máy chủ Backend nhận được yêu từ Load Balancer và xử lý nó để tạo ra một phản hồi cho máy khách.
- Chuyển tiếp phản hồi trở lại máy khách: Khi máy chủ Backend đã tạo ra một phản hồi nó sẽ gửi phản hồi đó trở lại Load Balancer. Load Balancer sau đó sẽ tiếp nhận phản hồi này và chuyển tiếp nó trở lại máy khách ban đầu.

1.1.2 Các phương pháp Load Balancing phổ biến

Có nhiều phương pháp Load Balancing khác nhau, mỗi phương pháp có những ưu điểm và nhược điểm riêng. Việc lựa chọn phương pháp phù hợp phụ thuộc vào yêu cầu cụ thể của ứng dụng và đặc điểm của các máy chủ Backend.

- Round Robin: Đây là một trong những thuật toán đơn giản nhất, phân phối các yêu cầu đến các máy chủ có sẵn theo một thứ tự luân phiên. Mỗi yêu cầu mới sẽ được gán cho máy chủ tiếp theo trong danh sách.
- Weighted Round Robin: Tương tự như Round Robin nhưng phương pháp này cho phép gán các trọng số khác nhau cho mỗi máy chủ dựa trên khả năng hoặc mức độ ưu tiên của chúng. Các máy chủ có trọng số cao hơn sẽ nhận được nhiều lưu lượng truy cập hơn.
- Least Connections: Thuật toán này chuyển hướng yêu cầu đến máy chủ có số lượng kết nối hoạt động ít nhất tại thời điểm đó. Điều này đảm bảo rằng các máy chủ đang xử lý ít tải hơn sẽ nhận được các yêu cầu mới, đặc biệt là trong các tình huống mà thời gian xử lý yêu cầu có thể khác nhau đáng kể. Phương pháp này thường được

khuyến nghị cho các phiên kết nối dài.

- IP Hash: Phương pháp này sử dụng một hàm băm để tạo ra một khóa dựa trên địa chỉ IP của máy khách. Khóa này sau đó được sử dụng để ánh xạ máy khách đến một máy chủ Backend cụ thể. Điều này đảm bảo rằng tất cả các yêu cầu từ một địa chỉ IP cụ thể sẽ luôn được gửi đến cùng một máy chủ, hữu ích cho các ứng dụng cần duy trì trạng thái phiên của người dùng trên một máy chủ cụ thể.
- Weighted Least Connection: Đây là một phương pháp kết hợp giữa Least Connections và Weighted Round Robin. Nó xem xét cả trọng số được gán cho máy chủ và số lượng kết nối hiện tại để quyết định máy chủ nào sẽ nhận yêu cầu tiếp theo.

Ngoài ra, còn có các thuật toán Load Balancing phức tạp hơn như Weighted Response Time, Software Defined Networking (SDN) Adaptive... Việc lựa chọn thuật toán phù hợp nhất đòi hỏi sự cân nhắc kỹ lưỡng về đặc điểm của ứng dụng, khả năng của máy chủ và mục tiêu hiệu suất mong muốn.

1.1.3 Các chế độ hoạt động của Load Balancer

Load Balancer hoạt động ở Layer 4 và Layer 7 theo mô hình OSI, tùy thuộc vào Layer hoạt động sẽ đem lại các lợi ích và cách hoạt động khác nhau:

- Layer 4 (Transport Layer): Load Balancer ở Layer 4 hoạt động tại tầng vận chuyển điều đó có nghĩa là chúng có thể đưa ra các quyết định định tuyến dựa trên các cổng TCP hoặc UDP mà các gói sử dụng cùng với địa chỉ IP nguồn và đích của chúng. Load Balancer Layer 4 thực hiện Network Address Translation nhưng không kiểm tra nội dung thực tế từng gói từ đó đem lại tốc độ cao hơn so với hoạt động ở Layer 7.
- Load Balancer Layer 7 (Application Layer): Load Balancer ở Layer 7 hoạt động tại tầng ứng dụng, cao nhất trong mô hình OSI. Chúng có thể đánh giá phạm vi dữ liệu rộng hơn, bao gồm các tiêu đề HTTP và ID phiên SSL, khi quyết định cách phân phối các yêu cầu trên toàn bộ cụm máy chủ. Load Balancing chuyên sâu hơn về mặt

tính toán tại Layer 7 hơn so với Layer 4 và nó cũng có thể hoạt động hiệu quả hơn ở Layer 7, do cấu hình được thêm vào để hiểu và xử lý các yêu cầu của máy khách đến máy chủ.

Load Balancer có thể hoạt động ở một số chế độ khác nhau, ảnh hưởng đến cách nó tương tác với máy khách và máy chủ Backend:

- Chế độ Bridge (Direct Server Return - DSR): Trong chế độ này, Load Balancer chỉ xử lý các yêu cầu đến và đưa ra quyết định định tuyến. Sau đó, nó chuyển tiếp yêu cầu trực tiếp đến máy chủ Backend mà không thay đổi địa chỉ đích. Máy chủ Backend sau đó sẽ trả lời trực tiếp cho máy khách, bỏ qua Load Balancer cho lưu lượng phản hồi. Chế độ này có thể cải thiện hiệu suất vì Load Balancer không phải xử lý lưu lượng phản hồi. Tuy nhiên, nó đòi hỏi cấu hình mạng phức tạp hơn. Chế độ Bridge thường hoạt động tại Layer 4.
- Chế độ NAT (Network Address Translation): Trong chế độ này, Load Balancer hoạt động như một proxy. Khi nó chuyển tiếp yêu cầu đến máy chủ Backend, nó thay đổi địa chỉ IP nguồn thành địa chỉ IP của chính nó (hoặc một địa chỉ IP khác trong mạng của nó). Máy chủ Backend sau đó sẽ gửi phản hồi trở lại Load Balancer, và Load Balancer sẽ thay đổi địa chỉ IP đích thành địa chỉ IP của máy khách ban đầu trước khi chuyển tiếp phản hồi. Đây là chế độ hoạt động phổ biến và đơn giản để cấu hình. Chế độ NAT thường hoạt động tại Layer 4.
- Chế độ Proxy đầy đủ: Trong chế độ này, Load Balancer thiết lập hai kết nối riêng biệt: một giữa máy khách và Load Balancer, và một giữa Load Balancer và máy chủ Backend. Load Balancer hoạt động như một trung gian hoàn toàn, kiểm tra và có thể sửa đổi cả yêu cầu và phản hồi. Chế độ này cung cấp sự linh hoạt cao nhất và cho phép các tính năng nâng cao như SSL termination và content-based routing. Chế độ Proxy đầy đủ cần hoạt động ở Layer 7 để có thể xử lý nội dung của các giao thức như HTTP, FTP… tuy nhiên nó vẫn có thể hoạt động ở Layer 4 nhưng sẽ không đem lại các hiệu quả hoàn chỉnh như tại Layer 7.

Việc tùy chọn Layer hay chế độ hoạt động của Load Balancer phụ thuộc vào nhiều yếu tố như mục đích sử dụng, quy mô máy chủ… từ đó sẽ đem lại hiệu quả khác nhau.

HAProxy chủ yếu hoạt động ở chế độ proxy đầy đủ cho cả Layer 4 (TCP Proxy) và Layer 7 (HTTP Proxy), cho phép nó cung cấp các tính năng Load Balancing và quản lý lưu lượng.

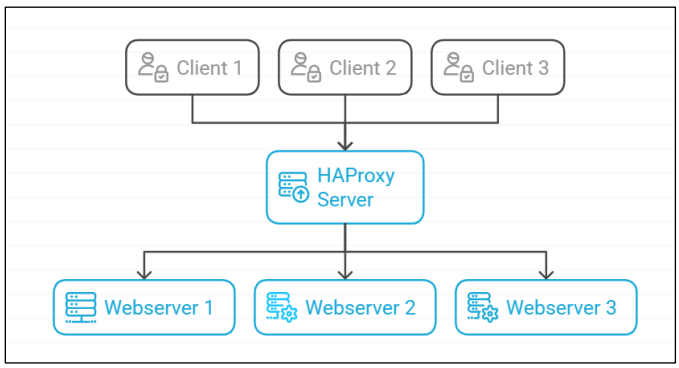
1.2. Giới thiệu về HAProxy

HAProxy (High Availability Proxy) là một phần mềm cân bằng tải và proxy ngược mã nguồn mở rất mạnh mẽ và đáng tin cậy. Nó được sử dụng rộng rãi để cải thiện hiệu suất và độ tin cậy của các ứng dụng Web bằng cách phân phối lưu lượng truy cập trên nhiều máy chủ.

Các chức năng chính của HAProxy:

- Cân bằng tải: HAProxy phân phối lưu lượng mạng đến trên nhiều máy chủ, ngăn ngừa tình trạng quá tải cho bất kỳ máy chủ riêng lẻ nào và đảm bảo rằng các yêu cầu được xử lý hiệu quả.
- Proxy ngược: HAProxy hoạt động như một trung gian giữa máy khách và máy chủ, che giấu danh tính của các máy chủ và cung cấp các tính năng bảo mật như SSL Termination.
- Tính khả dụng cao: HAProxy có thể tự động chuyển lưu lượng truy cập sang các máy chủ dự phòng trong trường hợp máy chủ gặp sự cố, đảm bảo rằng ứng dụng vẫn khả dụng.

1.3. Mô hình và cách hoạt động của HAProxy



Hình 1: Kiến trúc cân bằng tải HAProxy – Mô hình logic

Mô hình giả lập hoạt động HAProxy trong một máy chủ thực tế gồm các thành phần:

- Clients (Client 1, Client 2, ..., Client N): Đây là những người dùng hoặc ứng dụng gửi yêu cầu đến ứng dụng Web.
- HAProxy Server: Máy chủ này đóng vai trò là trung gian giữa các Client và các máy chủ Web. Nó nhận các yêu cầu đến và phân phối chúng đến các máy chủ Web khác nhau dựa trên thuật toán cân bằng tải đã định cấu hình.
- Webservers (Webserver 1, Webserver 2, Webserver 3): Đây là các máy chủ thực tế lưu trữ ứng dụng Web hoặc dịch vụ. HAProxy phân phối lưu lượng truy cập đến các máy chủ này để đảm bảo rằng không máy chủ nào bị quá tải.

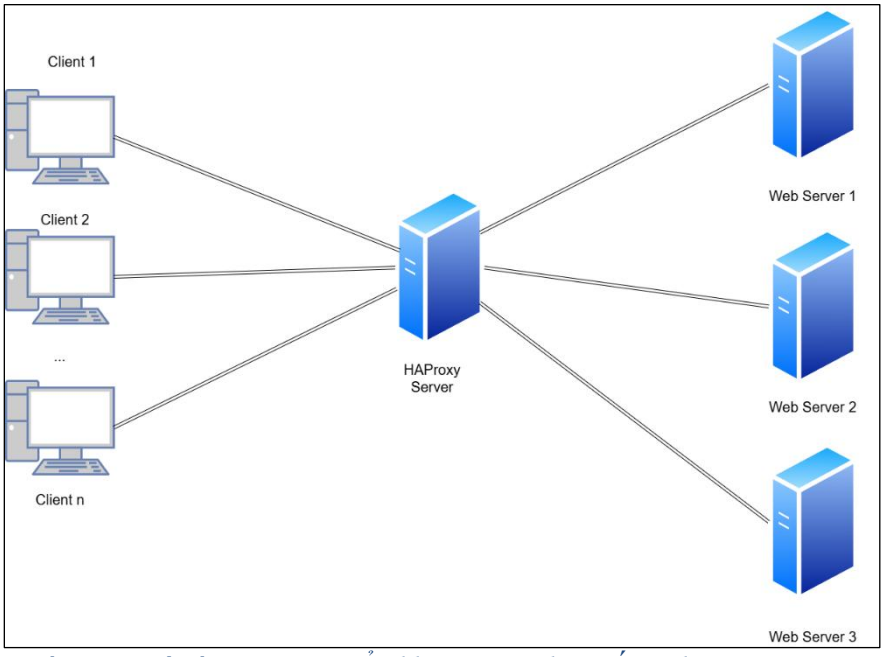
Cách thức hoạt động:

- Bước 1: Tiếp nhận yêu cầu: Khi một Client gửi một yêu cầu đến ứng dụng Web, yêu cầu này sẽ được gửi đến địa chỉ IP của máy chủ HAProxy.
- Bước 2: Quyết định cân bằng tải: Máy chủ HAProxy nhận được yêu cầu và sau đó quyết định máy chủ Web nào sẽ xử lý yêu cầu đó. Nó đưa ra quyết định này dựa trên thuật toán cân bằng tải đã được định cấu hình và tình trạng của các máy chủ Web.
- Bước 3: Chuyển tiếp yêu cầu: HAProxy chuyển tiếp yêu cầu đến máy chủ Web đã chọn.

- Bước 4: Xử lý yêu cầu và phản hồi: Máy chủ Web xử lý yêu cầu và gửi phản hồi trở lại máy chủ HAProxy.
- Bước 5: Chuyển tiếp phản hồi: HAProxy chuyển tiếp phản hồi từ máy chủ Web đến Client ban đầu.

PHẦN 2: TRIỂN KHAI

2.1. Mô hình



Hình 2: Mô hình mạng triển khai trong thực tế sử dụng HAProxy

Bảng 1: Bảng chi tiết thành phần trong mạng

STT	Địa chỉ IP (Sử dụng LAN ảo – thông qua Radmin VPN)	Vai trò
1	26.98.120.14	HAProxy Server & Web Server
2	26.51.240.125	Client 1
3	26.159.39.220	Client 2
4	26.23.1.43	Client 3

2.2. Các bước cài đặt HAProxy

Hướng dẫn cài đặt chi tiết: Windows | Docker Docs

- Bước 1: Cài đặt Docker về máy Windows, tiến hành tải về, cài đặt và khởi chạy.
- Bước 2: Cài đặt image HAProxy thông qua cửa sổ Terminal.

```
docker pull haproxy
```

```
PS C:\Users\Tofu> docker pull haproxy
Using default tag: latest
latest: Pulling from library/haproxy
Digest: sha256:5faf31607d4333d02379f8642628edece8770fb428d90917f18bf3eec7bd2d84
```

Hình 3: Tải Docker Image cho HAProxy từ Docker Hub

- Bước 3: Để tiến hành cài đặt cấu hình HAProxy, tạo một file haproxy.cfg (tại một đường dẫn thích hợp) để đảm bảo có thể đảm bảo chạy được container HAProxy.

2.3. Cấu hình HAProxy trên Layer 4

- HAProxy và WebServer sẽ được đặt tại cùng 1 máy chủ có cùng địa chỉ IP (26.98.120.14) nhưng sẽ hoạt động tại các port khác nhau.
- Mỗi khi nhận được 1 yêu cầu từ Client HAProxy sẽ tiếp nhận yêu cầu đó và chuyển nó tới 1 trong 3 Webserver đã được cấu hình sẵn.

Các bước cấu hình:

- Bước 1: Tạo một dạng giao diện Web đơn giản với HTML:

Hình 4: Giao diện phản hồi Web Server 1 thông qua HAProxy

```html
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Web Server #1</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      background-color: #f0f2f5;
      padding: 50px;
    }
    h1 {
      color: #2c3e50;
    }
    h2 {
      color: #2980b9;
      margin-top: 20px;
    }
    img {
      margin-top: 30px;
      width: 200px;
    }
  </style>
</head>
<body>
  <h1>Web server #1 đang phục vụ bạn!</h1>
  <h2>Mạng máy tính và truyền thông - HAProxy</h2>
  <img src="Logo_UIT_updated.svg" alt="UIT Logo">
</body>
</html>
```

- Thay đổi nội dung hiển thị để phân biệt được giữa các Webserver khác nhau.

- Bước 2: Triển khai các Web Server trên Docker với Nginx – alpine thông qua Dockerfile, phiên bản nhẹ alpine sẽ phù hợp với các Web không quá phức tạp.

- Cấu hình file nginx.conf để thiết lập lắng nghe trên cổng 8081 (nếu như không thiết lập nginx.conf, mặc định Nginx sẽ lắng nghe trên cổng 80).

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    sendfile on;
    keepalive_timeout 65;

    server {
        listen 8081; # Cấu hình Nginx lắng nghe trên cổng 8081
        server_name _;

        location / {
            root /usr/share/nginx/html;
            index index.html;
        }
    }
}
```

- Cấu hình File Dockerfile

```
# Sử dụng Nginx nhẹ
FROM nginx:alpine

# Xóa cấu hình Nginx mặc định
RUN rm /etc/nginx/conf.d/default.conf

# Sao chép cấu hình Nginx tùy chỉnh từ thư mục build hiện tại
COPY nginx.conf /etc/nginx/nginx.conf

# Copy file index.html vào thư mục của Nginx
COPY index.html /usr/share/nginx/html/index.html
# Copy logo
COPY Logo_UIT_updated.svg
/usr/share/nginx/html/Logo_UIT_updated.svg

# Mở cổng 8081 để phục vụ web
EXPOSE 8081

# Lệnh chạy Nginx với cấu hình mới
CMD ["nginx", "-g", "daemon off;"]
```

- Bước 3: Cấu hình cho cân bằng tải Haproxy thông qua file haproxy.cfg.

    - Cấu hình file haproxy.cfg

```
defaults
    log global # Ghi log toàn cục
    mode tcp # Chế độ TCP
    option tcplog # Ghi log TCP
    option dontlognull # Không ghi log cho các kết nối không có dữ
liệu
    timeout connect 5000 # Thời gian chờ kết nối tối đa
    timeout client  50000 # Thời gian chờ của client tối đa
    timeout server  50000 # Thời gian chờ của server tối đa


front tcp_front
    bind *:8081 # Lắng nghe trên cổng 8081
    default_backend Web_servers # Sử dụng backend Web_servers làm mặc
định

backend Web_servers
    balance roundrobin # Phân phối theo giải thuật Round Robin
    # Trỏ tới các container
    server Web1 Web1:8081 check # Kiểm tra sức khỏe của server Web1
    server Web2 Web2:8081 check # Kiểm tra sức khỏe của server Web2
    server Web3 Web3:8081 check # Kiểm tra sức khỏe của server Web3
```

- Bước 4: Cấu hình file docker-compose.yml để có thể chạy trên docker:

```
services:
  haproxy:
    image: haproxy:alpine
    container_name: haproxy_layer4
    restart: always
    volumes:
      - ./haproxy/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    ports:
      - "8000:8081" #Phân phối cổng layer 4 cho TCP
    depends_on:
      - Web1
      - Web2
      - Web3

  Web1:
    build: ./Webserver/Web1
    container_name: Web1
    restart: always

  Web2:
    build: ./Webserver/Web2
    container_name: Web2
    restart: always

  Web3:
    build: ./Webserver/Web3
    container_name: Web3
    restart: always
```

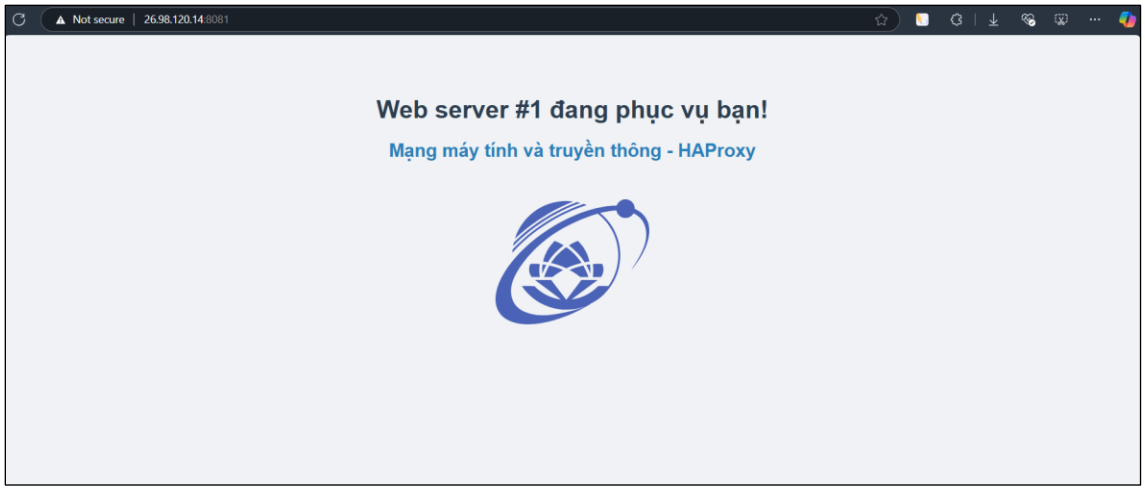
- Bước 5: Sử dụng lệnh sau trong powershell để chạy HAProxy và các Webserver đã cấu hình:

```
docker-compose up -d –build
```

2.3.1. Thử nghiệm với HAProxy Layer 4

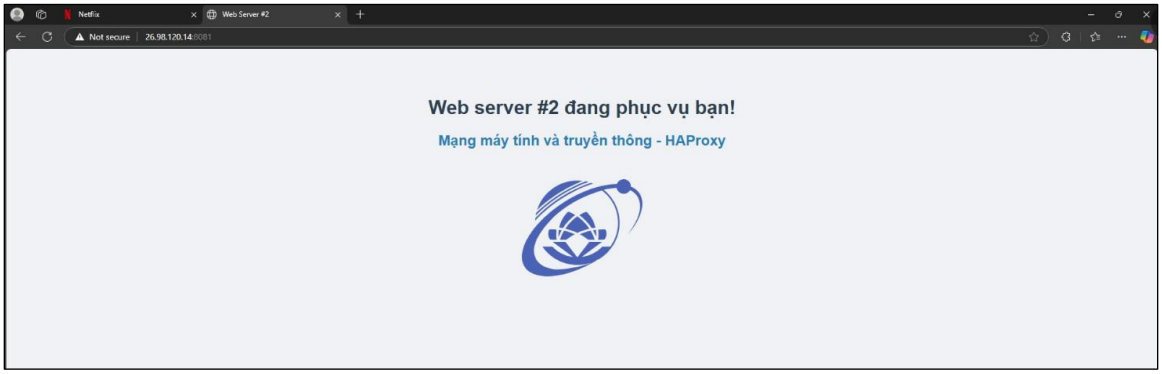
Ba Client sẽ gửi request tới Server HAProxy cùng 1 lúc:

- Kết quả từ Client 1:



Hình 5: Kết quả Client 1 gửi request tới Server HAProxy

- Kết quả từ Client 2:



Hình 6: Kết quả Client 2 gửi request tới Server HAProxy

- Kết quả từ Client 3:

Hình 7: Kết quả Client 3 gửi request tới Server HAProxy

- Phản hồi từ HAProxy cho thấy đã phân phối các yêu cầu ra các Webserver 1, Webserver 2, Webserver 3:

```
172.18.0.1:59624 [06/Apr/2025:13:45:30.308] tcp_front web_servers/Web1 1/0/50124 9411 cD 1/1/0/0/0 0/0
172.18.0.1:39652 [06/Apr/2025:13:50:56.922] tcp_front web_servers/Web2 1/0/50627 9411 cD 2/2/1/0/0 0/0
172.18.0.1:49642 [06/Apr/2025:13:51:42.223] tcp_front web_servers/Web3 1/0/30353 869 -- 1/1/0/0/0 0/0
```

Hình 8: HAProxy phân phối tải đến các Web Server qua log truy cập TCP

- Giải thích: Dù truy cập chung 1 địa chỉ IP nhưng HAProxy đã phân phối các lưu lượng khác nhau nên các Client cũng nhận được phản hồi từ các Webserver khác nhau.

- Trường hợp tắt 1 Webserver (Webserver 1) để giả lập trường hợp server có vấn đề trục trặc.

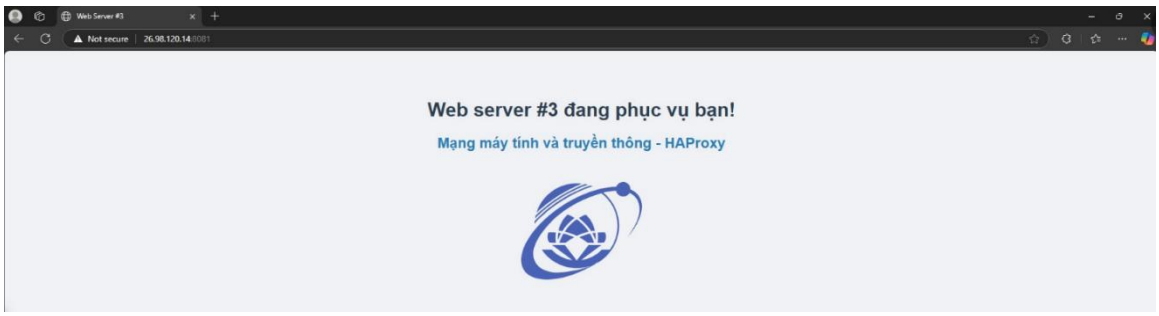
  - Sử dụng Docker Desktop để tắt đi Webserver 1:

Hình 9: Mô phỏng sự cố Web Server bằng cách tắt đi Web1 trong Docker Desktop

  - Kết quả từ Client 1:

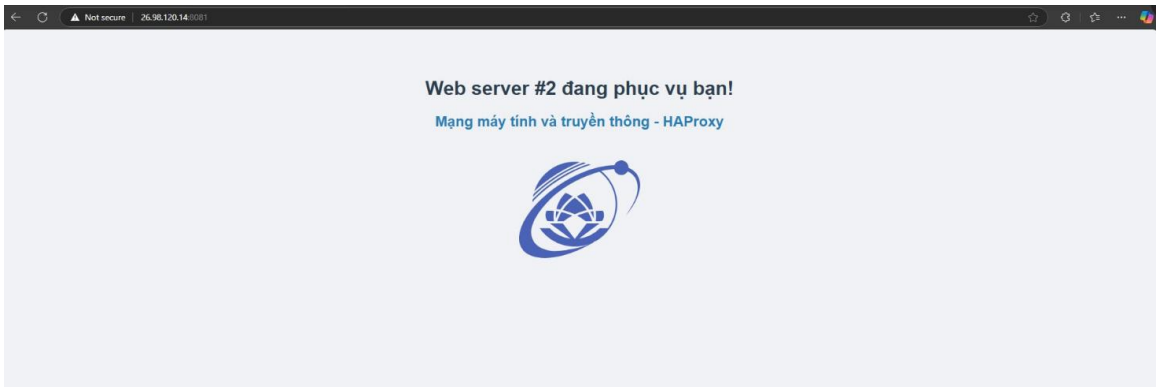
Hình 10: Client 1 được chuyển tiếp tới Web Server 2 sau khi ngừng hoạt động Web Server 1

- Kết quả từ Client 2:



Hình 11: Client 2 được chuyển tiếp tới Web Server 3 sau khi ngừng hoạt động Web Server 1

- Kết quả từ Client 3:



Hình 12: Client 3 được chuyển tiếp tới Web Server 2 sau khi ngừng hoạt động Web Server 1

- Kết quả từ HAProxy cho thấy đã phân phối lưu lượng tới các Webserver còn hoạt động (Webserver 2, Webserver 3):

```
172.18.0.1:44386 [06/Apr/2025:14:11:45.503] tcp_front web_servers/Web2 1/0/14828 2160 -- 2/2/1/0/0 0/0
172.18.0.1:45440 [06/Apr/2025:14:12:00.415] tcp_front web_servers/Web3 1/0/90 869 -- 2/2/1/1/0 0/0
172.18.0.1:58622 [06/Apr/2025:14:12:00.911] tcp_front web_servers/Web2 1/0/0 869 -- 2/2/1/0/0 0/0
```

Hình 13: HAProxy phân phối tải tới Web Server 2 và 3 thông qua log truy cập TCP

Giải thích: Vì Webserver 1 đã gặp trục trặc nên HAProxy sẽ tự động phân phối lưu lượng truy cập tới 2 Webserver còn lại.

2.4. Cấu hình HAProxy trên Layer 7: HTTP

- Bước 1: Cấu hình trên file .cfg vừa tạo.

```
global
    log stdout format raw daemon #Ghi log ra stdout
    maxconn 2000 #Giới hạn kết nối tối đa
    daemon #Chạy dưới dạng nền

defaults
    log     global
    mode    http #Layer 7 mode
    option  httplog #Ghi log
    option  dontlognull #Không ghi log cho các kết nối không có dữ
liệu
    timeout connect 5000 #Thời gian chờ kết nối tối đa
    timeout client  50000 #Thời gian chờ của client tối đa
    timeout server  50000 #Thời gian chờ của server tối đa

frontend http_front #Phần đầu vào HTTP
    bind *:80 #Lắng nghe trên cổng 80
    default_backend Web_servers
#Sử dụng backend Web_servers làm mặc định

backend Web_servers
    balance roundrobin #Phân phối theo giải thuật Round Robin
    option httpchk GET
#Kiểm tra sức khỏe của server bằng cách gửi yêu cầu GET tới "/"

#Trỏ tới các container
    server Web1 Web1:80 check #Kiểm tra sức khỏe của server Web1
    server Web2 Web2:80 check #Kiểm tra sức khỏe của server Web2
    server Web3 Web3:80 check #Kiểm tra sức khỏe của server Web3
```

- Bước 2: Cấu hình các Webserver để trả về kết quả khác nhau ở mỗi Web. Các Webserver sẽ trả về nội dung tương ứng với số thứ tự của chúng:

Hình 14: Giao diện các Web Server

- Bước 3: Đóng gói và chạy qua docker bằng câu lệnh:

```
docker-compose up -d
```

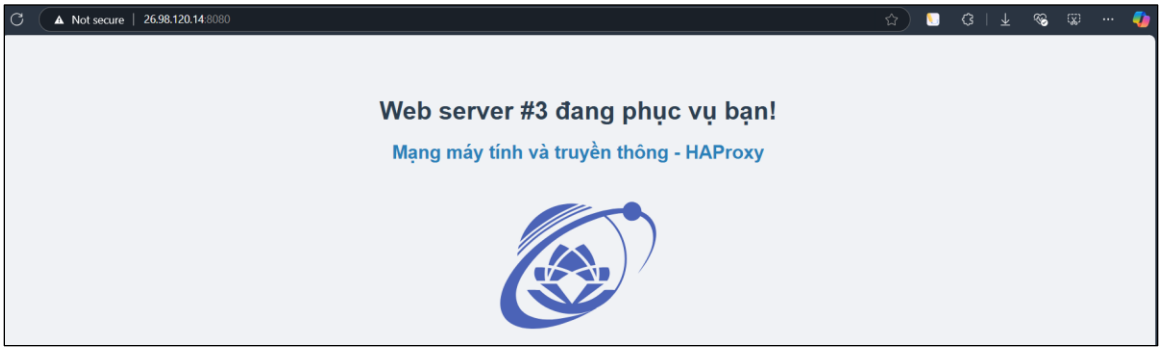
2.4.1. Thử nghiệm với HAProxy Layer 7

Với HAProxy Layer 7 sẽ luân phiên phân phối yêu cầu giữa các Webserver khác nhau. Cứ theo một khoảng thời gian được cấu hình trước (20 giây) thì Webserver được chọn sẽ thay đổi sang một Webserver khác sẵn sàng, thứ tự ưu tiên của các Webserver được chọn dựa trên việc Webserver nào có lượng công việc ít nhất tại thời điểm đấy.

Kết quả khi thử tại cùng 1 Client từ các lần yêu cầu truy xuất khác nhau:

Hình 15: Client gửi request đến Server HAProxy lần đầu tiên

Hình 16: Client gửi request đến Server HAProxy lần thứ hai



Hình 17: Client gửi request đến Server HAProxy lần thứ ba

Kết quả trả về từ HAProxy: Cho thấy dù chỉ có 1 Client kết nối với server tại 1 thời điểm nhưng HAProxy vẫn chia các yêu cầu từ Client đấy đến từng server khác nhau.

Với Layer 7, HAProxy còn có thể hiểu được giao thức HTTP, SSL/TLS, FTP,… HAProxy sẽ cân bằng tải dựa trên nội dung của yêu cầu như HTTP Metadata, IP nguồn,

IP đích, cổng.

Với tuỳ chọn “option httpchk GET” có thể kiểm tra được tình trạng của các Web Server rằng nó có hoạt động ổn định hay không.

```
web1  | 172.19.0.5 - - [04/May/2025:07:47:42 +0000] "GET / HTTP/1.0" 200 0 "-" "-" "-"
web2  | 172.19.0.5 - - [04/May/2025:07:47:43 +0000] "GET / HTTP/1.0" 200 0 "-" "-" "-"
web3  | 172.19.0.5 - - [04/May/2025:07:47:44 +0000] "GET / HTTP/1.0" 200 0 "-" "-" "-"
web1  | 172.19.0.5 - - [04/May/2025:07:47:44 +0000] "GET / HTTP/1.0" 200 631 "-" "-" "-"
web2  | 172.19.0.5 - - [04/May/2025:07:47:45 +0000] "GET / HTTP/1.0" 200 631 "-" "-" "-"
web3  | 172.19.0.5 - - [04/May/2025:07:47:46 +0000] "GET / HTTP/1.0" 200 631 "-" "-" "-"
web1  | 172.19.0.5 - - [04/May/2025:07:47:46 +0000] "GET / HTTP/1.0" 200 631 "-" "-" "-"
web2  | 172.19.0.5 - - [04/May/2025:07:47:47 +0000] "GET / HTTP/1.0" 200 631 "-" "-" "-"
```

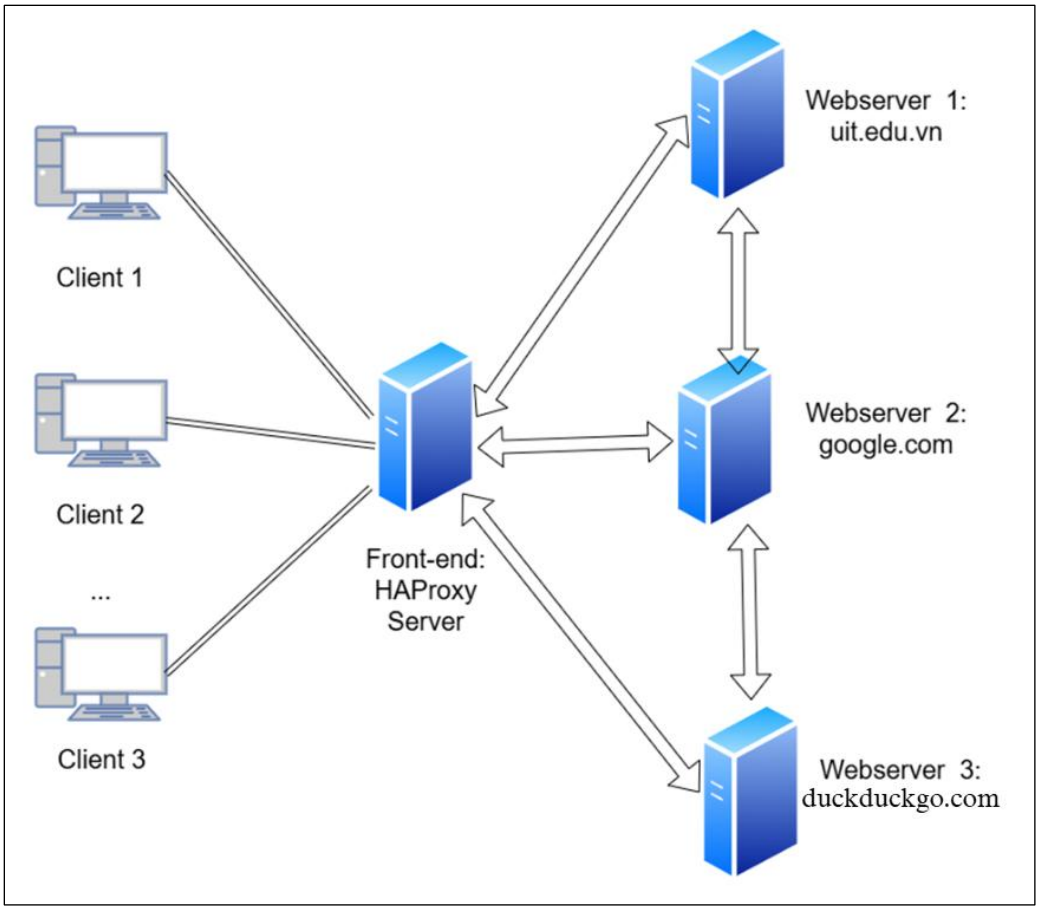
Hình 18: Log truy cập HTTP từ Client đến Backend Server – Mã trạng thái 200

```
[WARNING]  (8) : Server webserver/server3 is DOWN, reason: Layer7 wrong sta
tus, code: 400, info: "Bad Request", check duration: 134ms. 2 active and 0
backup servers left. 0 sessions active, 0 requeued, 0 remaining in queue.
Server webserver/server3 is DOWN, reason: Layer7 wrong status, code: 400, i
nfo: "Bad Request", check duration: 134ms. 2 active and 0 backup servers le
ft. 0 sessions active, 0 requeued, 0 remaining in queue.
```

Hình 19: Log truy cập HTTP thông báo rằng Server bị lỗi với mã trạng thái là 400

2.5. Trên Layer 7, chỉnh cấu hình HAProxy để có thể kết nối giữa HAProxy và các Client theo chuẩn mã hoá SSL/TLS.

Mô hình sử dụng để cấu hình HAProxy, thay thế sử dụng các Webserver thành các trang web công cộng để đánh giá độ tin cậy của việc sử dụng chuẩn mã hoá SSL/TLS với Clients:



Hình 20: Mô hình triển khai thực tế trong HAProxy với giao thức SSL/TLS tại Layer 7

Đối với trường hợp sử dụng tên miền ảo (Local Host), có thể sử dụng một chứng chỉ tự ký (self-signed certificate) để tạo chứng chỉ cục bộ.

- Bước 1: Cài đặt OpenSSL trên Windows.

Đường dẫn: Win32/Win64 OpenSSL Installer for Windows - Shining Light Productions

- Bước 2: Mở OpenSSL và thực hiện câu lệnh sau.

```
openssl req -x509 -newkey rsa:2048 -keyout testhaproxy1.key -out testhaproxy1.crt -days 365 -nodes -subj "/CN=testhaproxy1.com" -addext "subjectAltName=DNS:testhaproxy1.com,DNS:testhaproxy2.com,DNS:testhaproxy3.com"
```

- Trong đó:
  - req -x509: Chứng chỉ tự ký.
  - -newkey rsa:2048: Khoá RSA-2048 bit.
  - -keyout testhaproxy1.key: file khoá riêng.
  - -out testhaproxy1.crt: file chứng chỉ.
  - -days 365: Chứng chỉ hiệu lực trong 365 ngày.
  - -nodes: Không mã hoá khoá riêng (bỏ qua mật khẩu)
  - -subj "/CN=testhaproxy1.com": Đặt Common Name (CN) là testhaproxy1.com
  - -addext "subjectAltName=…": Thêm Subject Alternative Name để hỗ trợ đa tên miền.
- Bước 3: Tiến hành tạo file kết hợp chứng chỉ và khoá riêng cho file ".pem" cho HAProxy.

```
copy testhaproxy1.crt + testhaproxy1.key haproxy-cert.pem
```

  - File .pem được tạo tại đường dẫn khi thực hiện câu lệnh.
- Bước 4: Cấu hình file .cfg để sử dụng chứng chỉ SSL trên Layer 7: HTTP mode.

```
global
  maxconn 4096 # Giới hạn kết nối tối đa
  daemon # Chạy dưới dạng nền
  log stdout format raw daemon  # Sửa log để xuất ra stdout, phù
hợp với Docker

defaults
    timeout connect 10s # Thời gian chờ kết nối tối đa
    timeout client 500s # Thời gian chờ của client tối đa
   timeout server 500s # Thời gian chờ của server tối đa
    log global # Ghi log toàn cục
    mode http # Layer 7 mode
    maxconn 3000 # Giới hạn kết nối tối đa
    stats enable # Bật tính năng thống kê
    stats uri /haproxy_stats/acl # Đường dẫn tới trang thống kê
    stats auth admin:admin # Thông tin xác thực cho trang thống
kê
    stats refresh 30s # Làm mới trang thống kê mỗi 10 giây
    stats realm Haproxy ACL\ Statistics # Tiêu đề cho trang thống
kê
    frontend http-in
    bind *:80 # Lắng nghe trên cổng 80
    redirect scheme https if !{ ssl_fc }  # Chuyển hướng HTTP
sang HTTPS

frontend https-in
bind *:443 ssl crt /usr/local/etc/haproxy/certs/haproxy-cert.pem
# Lắng nghe trên cổng 443 với SSL
mode http # Chế độ HTTP
option httplog  # Thêm log chi tiết cho HTTP
default_backend Webserver  # Thêm backend mặc định

backend Webserver
mode http # Chế độ HTTP
balance roundrobin # Phân phối theo giải thuật Round Robin
option httpchk GET / # Kiểm tra sức khỏe của server bằng cách gửi
yêu cầu GET tới "/"
option httpclose # Đóng kết nối sau khi hoàn thành yêu cầu
option forwardfor # Thêm header X-Forwarded-For vào yêu cầu
```

```
server server1 uit.edu.vn:443 check ssl verify none  # server
uit.edu.vn:443 với SSL
server server2 opera.com:443 check ssl verify none # server
google.com:443 với SSL
server server3 duckduckgo.com:443 check ssl verify none # server
wikipedia.org:443 với SSL
```

- Bước 5: Tiến hành chạy file .cfg vừa cấu hình với Terminal.

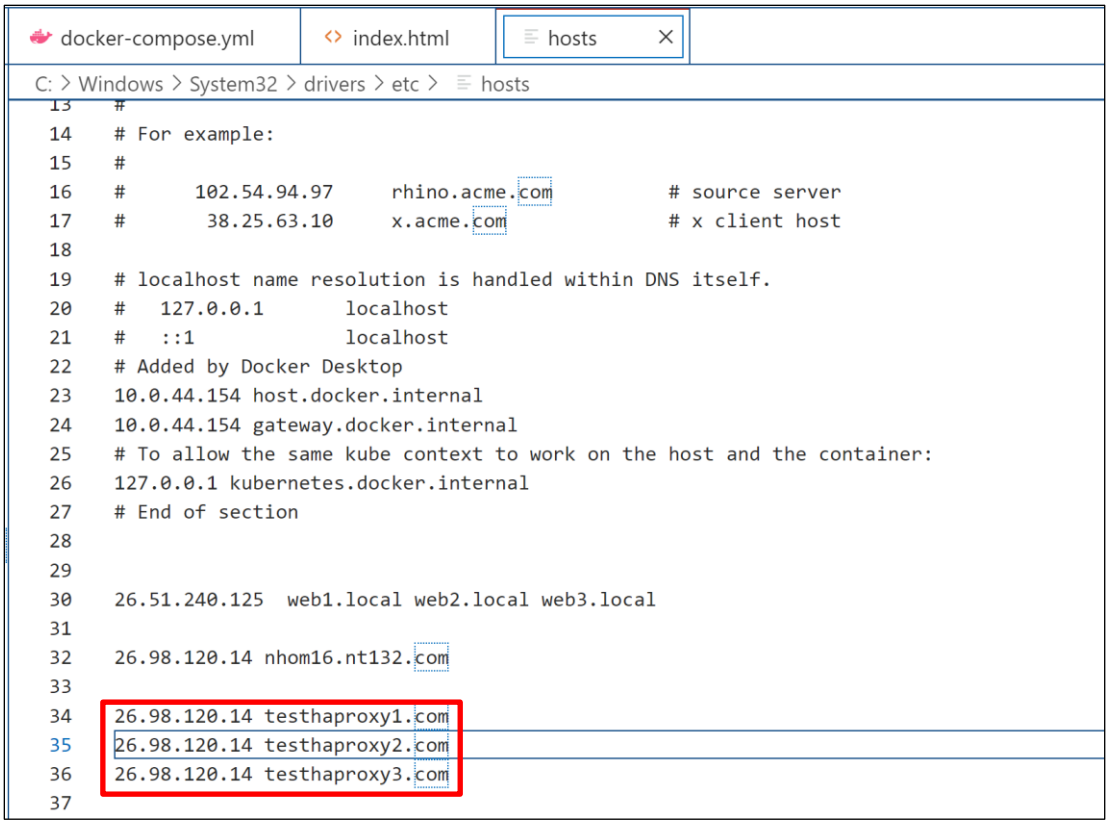
```
docker run -v
D:\Proxy\testacl.cfg:/usr/local/etc/haproxy/haproxy.cfg -v
D:\Proxy\haproxy-cert.pem:/usr/local/etc/haproxy/certs/haproxy-
cert.pem -p 443:443 -p 8080:80 haproxy
```

Trong đó:

- -v D:\Proxy\testacl.cfg:/usr/local/etc/haproxy/haproxy.cfg: Ánh xạ file đã tiến hành cấu hình HAProxy vào trong container Docker.
- -v :\Proxy\haproxy-cert.pem:/usr/local/etc/haproxy/certs/haproxy-cert.pem: Ánh xạ file chứng chỉ SSL/TLS vào trong container.
- -p 443:443: Ánh xạ cổng 443 (HTTPS) vào container.
- -p 8080:80: Ánh xạ cổng 80 vào cổng 8080 trên máy cần.
- haproxy: image của Docker (mặc định là phiên bản mới nhất).

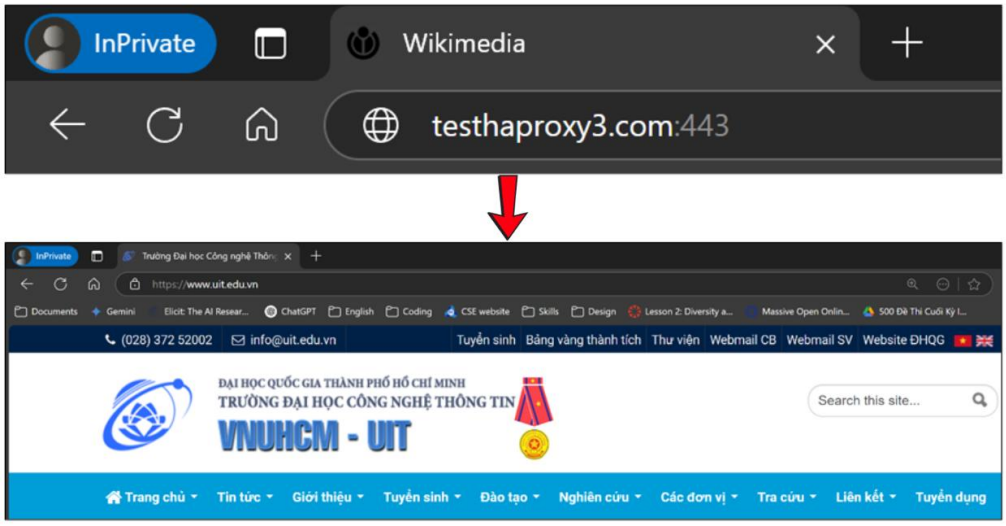
Kiểm tra kết quả:

- Địa chỉ testhaproxy3.com là địa chỉ đặt tên miền nội bộ của máy chủ chứa Proxy Server (26.98.120.14).



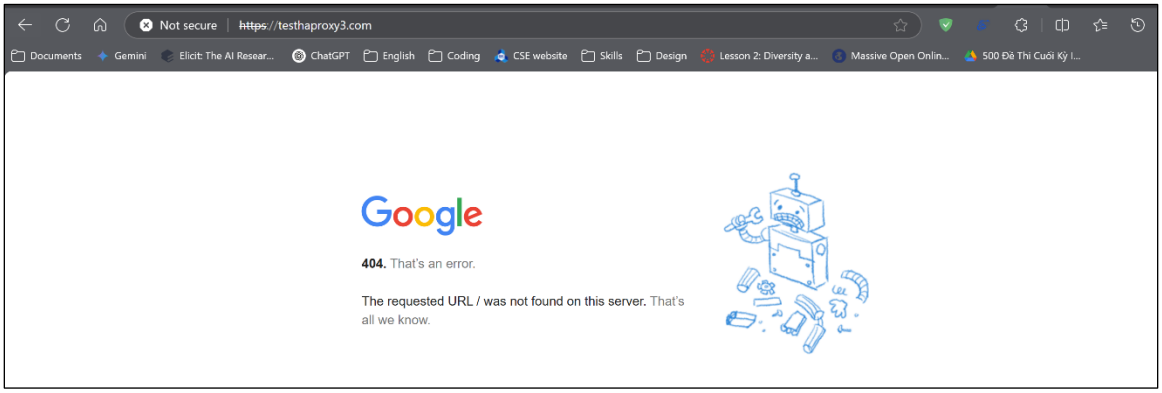
Hình 21: Cấu hình ánh xạ tên miền cho các Webserver trong file host (Windows)

- Truy cập vào https://testhaproxy3.com:443



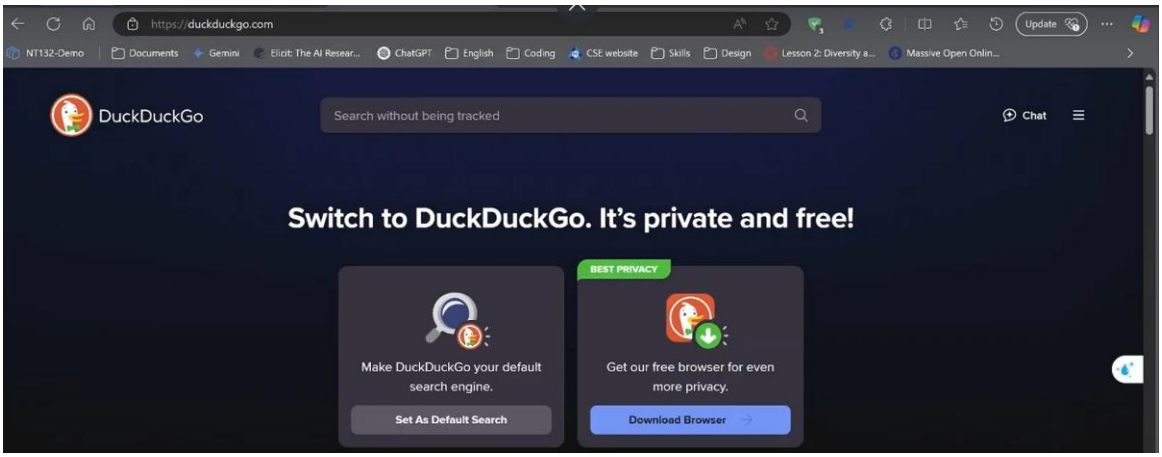
Hình 22: Client gửi yêu cầu đến Server HAProxy, sau đó được chuyển tiếp tới trang uit.edu.vn:443

- Truy cập vào https://testhaproxy3.com:443 lần 2



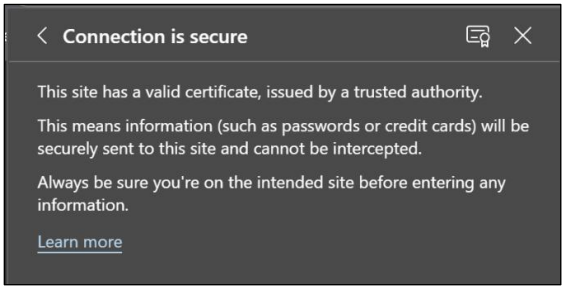
Hình 23: Client gửi yêu cầu đến Server HAProxy, sau đó được chuyển tiếp tới trang google.com:443

- Truy cập vào https://testhaproxy3.com:443 lần 3



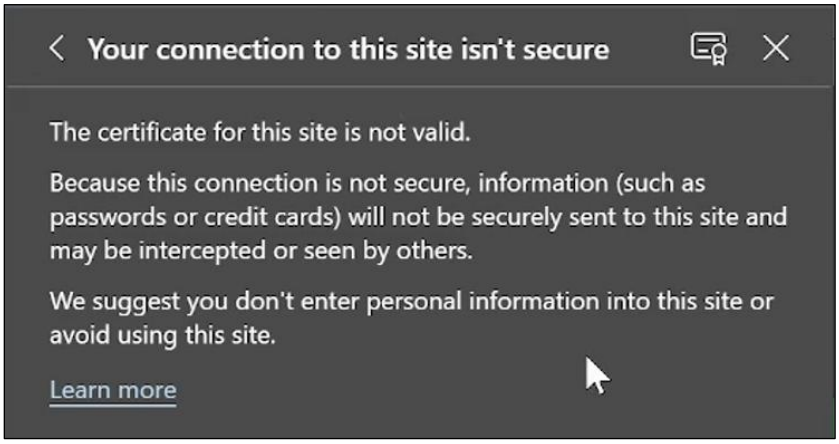
Hình 24: Client gửi yêu cầu đến Server HAProxy, sau đó được chuyển tiếp tới trang duckduckgo.com:443

- Chứng chỉ SSL/TLS hiệu lực với máy chủ: uit.edu.vn; duckduckgo.com.



Hình 25: Đối với một số trang web không yêu cầu xác thực chứng chỉ tự kí - kết nối an toàn

- Đối với một số phần lớn trang web công cộng như google.com. Việc sử dụng chứng chỉ tự ký được coi là không đáng tin cậy vì không có bên thứ ba (Certificate Authority) nào xác nhận danh tính của trang web.



Hình 26: Chứng chỉ không được chấp nhận - cần xác thực từ bên CA thứ ba uy tín

- Kết quả từ Terminal:

```
PS C:\Users\Tofu> docker run -v D:\Proxy\testacl.cfg:/usr/local/etc/haproxy/haproxy.cfg -v D:\Proxy\haproxy-cert.pem:/usr/local/etc/haproxy/certs/ha
proxy-cert.pem -p 443:443 -p 8080:80 haproxy
[NOTICE]   (1) : Initializing new worker (8)
[NOTICE]   (1) : Loading success.
172.17.0.1:41544 [06/Apr/2025:14:45:00.420] https-in/1: SSL handshake failure (error:0A000416:SSL routines::sslv3 alert certificate unknown)
172.17.0.1:41546 [06/Apr/2025:14:45:00.442] https-in~ webserver/server1 0/0/96/38/134 302 103 - - ---- 1/1/0/0/0 0/0 "GET https://testhaproxy3.com/
HTTP/2.0"
172.17.0.1:41546 [06/Apr/2025:14:45:04.892] https-in~ webserver/server2 0/0/146/162/308 404 1768 - - ---- 1/1/0/0/0 0/0 "GET https://testhaproxy3.co
m/ HTTP/2.0"
172.17.0.1:41546 [06/Apr/2025:14:45:13.766] https-in~ webserver/server3 0/0/284/108/392 301 410 - - ---- 1/1/0/0/0 0/0 "GET https://testhaproxy3.com
/ HTTP/2.0"
Connect from 172.17.0.1:55650 to 172.17.0.2:80 (http-in/HTTP)
[NOTICE]   (1) : haproxy version is 3.1.6-d929ca2
[NOTICE]   (1) : path to executable is /usr/local/sbin/haproxy
[WARNING]  (1) : Exiting Master process...
[ALERT]    (1) : Current worker (8) exited with code 130 (Interrupt)
[WARNING]  (1) : All workers exited. Exiting... (130)
```

Hình 27: Log phân tích chỉ ra chứng chỉ không được xác thực

2.6. Triển khai HAProxy ACL để phân phối các yêu cầu tới các máy chủ.

2.6.1. Access Control List

- Access Control List (ACL) trong HAProxy được sử dụng để kiểm tra một danh sách các điều kiện, sau đó tiến hành các quyết định khác nhau như:

  - Từ chối một yêu cầu
  - Chọn một Backend phù hợp dựa trên ACL
  - Thao tác với header HTTP, viết lại URL trước khi gửi tới phía Backend.

- Thao tác chuyển hướng.

2.6.2. Cú pháp khai triển một Access Control List

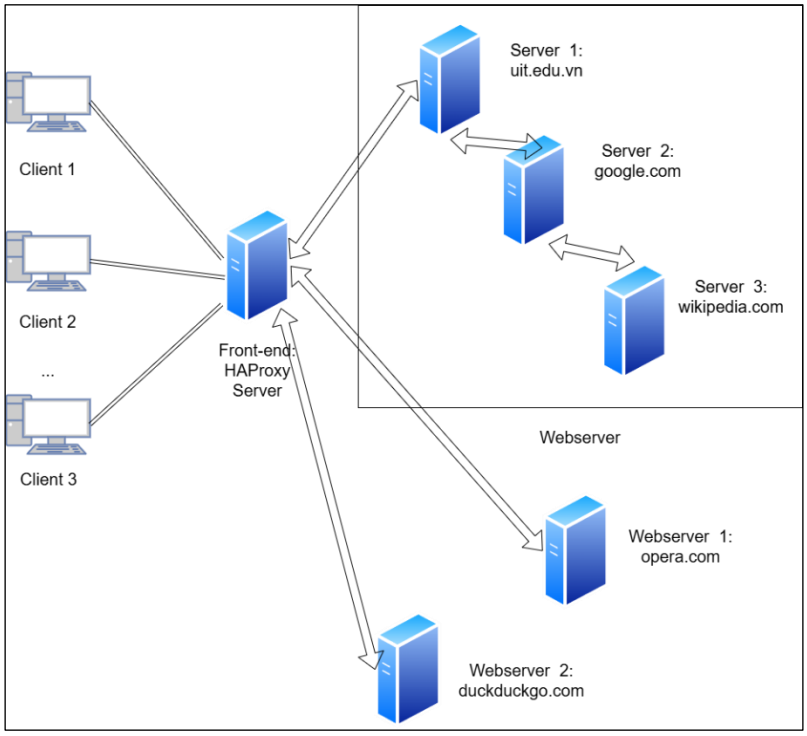
```
acl <acl_name> <criterion> [<operator> <value>]
```

Trong đó

- acl: định nghĩa một ACL.
- <acl_name>: tên định danh cho ACL được người dùng định nghĩa.
- <criterion>: tiêu chí kiểm tra như path_beg, hdr, src, method,..
- <operator>: các toán tử (-m, -i,..) – không bắt buộc.
- <value>: giá trị - không bắt buộc.

2.6.3. Áp dụng ACL vào cấu hình HAProxy

- Bước 1: Sử dụng .cfg đã sử dụng từ phần 2.5 để tái sử dụng nâng cấp mô hình.



Hình 28: Mô hình triển khai thực tế đối với HAProxy - ACL

```
global
    maxconn 4096 # Giới hạn kết nối tối đa
    daemon # Chạy dưới dạng nền
    log stdout format raw daemon  # Sửa log để xuất ra stdout,
phù hợp với Docker

defaults
    timeout connect 10s # Thời gian chờ kết nối tối đa
    timeout client 500s # Thời gian chờ của client tối đa
    timeout server 500s # Thời gian chờ của server tối đa
    log global # Ghi log toàn cục
    mode http # Layer 7 mode
    maxconn 3000 # Giới hạn kết nối tối đa
    stats enable # Bật tính năng thống kê
    stats uri /haproxy_stats/acl # Đường dẫn tới trang thống kê
    stats auth admin:admin # Thông tin xác thực cho trang thống
kê
    stats refresh 30s # Làm mới trang thống kê mỗi 10 giây
    stats realm Haproxy ACL\ Statistics # Tiêu đề cho trang thống
kê

frontend http-in
    bind *:80 # Lắng nghe trên cổng 80
    redirect scheme https if !{ ssl_fc }  # Chuyển hướng HTTP
sang HTTPS

frontend https-in
    bind *:443 ssl crt /usr/local/etc/haproxy/certs/haproxy-
cert.pem # Lắng nghe trên cổng 443 với SSL
    mode http # Chế độ HTTP
    option httplog  # Thêm log chi tiết cho HTTP
    acl acl_test1 hdr(host) -i testhaproxy1.com # Kiểm tra tên
miền trong header HTTP
    acl acl_test2 hdr(host) -i testhaproxy2.com # Kiểm tra tên
miền trong header HTTP
    acl acl_test3 hdr(host) -i testhaproxy3.com # Kiểm tra tên
miền trong header HTTP
    use_backend Webserver if acl_test1
    use_backend Webserver1 if acl_test2
```

```
    use_backend Webserver2 if acl_test3 # Sử dụng backend
Webserver nếu tên miền khớp
    default_backend Webserver  # Thêm backend mặc định

backend Webserver
    mode http # Chế độ HTTP
    balance roundrobin # Phân phối theo giải thuật Round Robin
    option httpchk GET / # Kiểm tra sức khỏe của server bằng cách
gửi yêu cầu GET tới "/"
    option httpclose # Đóng kết nối sau khi hoàn thành yêu cầu
    option forwardfor # Thêm header X-Forwarded-For vào yêu cầu
    server server1 uit.edu.vn:443 check ssl verify none  # server
uit.edu.vn:443 với SSL
    server server2 google.com:443 check ssl verify none # server
google.com:443 với SSL
    server server3 wikipedia.org:443 check ssl verify none #
server wikipedia.org:443 với SSL

backend Webserver1
    mode http # Chế độ HTTP
    option httpchk GET / # Kiểm tra sức khỏe của server bằng cách
gửi yêu cầu GET tới "/"
    option httpclose # Đóng kết nối sau khi hoàn thành yêu cầu
    option forwardfor # Thêm header X-Forwarded-For vào yêu cầu
    server server1 opera.com:443 check ssl verify none

backend Webserver2
    mode http # Chế độ HTTP
    option httpchk GET / # Kiểm tra sức khỏe của server bằng cách
gửi yêu cầu GET tới "/"
    option httpclose # Đóng kết nối sau khi hoàn thành yêu cầu
    option forwardfor # Thêm header X-Forwarded-For vào yêu cầu
    server server1 duckduckgo.com:443 check ssl verify none
```

- Bước 2: Chạy container Docker file vừa cấu hình.

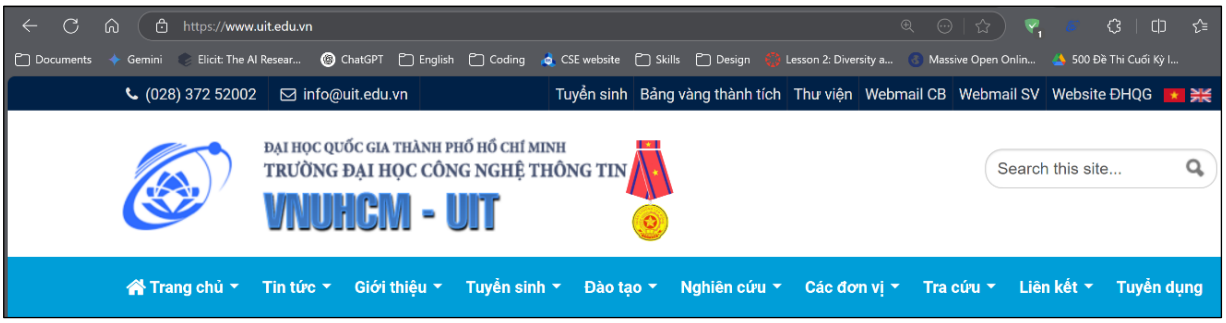
```
docker run -v
D:\Proxy\testacl2.cfg:/usr/local/etc/haproxy/haproxy.cfg -v
D:\Proxy\haproxy-cert.pem:/usr/local/etc/haproxy/certs/haproxy-
cert.pem -p 443:443 -p 8080:80 haproxy
```

Trong đó:

- -v D:\Proxy\testacl2.cfg:/usr/local/etc/haproxy/haproxy.cfg: Ánh xạ file đã tiến hành cấu hình HAProxy với ACL vào trong container Docker.
- -v :\Proxy\haproxy-cert.pem:/usr/local/etc/haproxy/certs/haproxy-cert.pem: Ánh xạ file chứng chỉ SSL/TLS vào trong container.
- -p 443:443: Ánh xạ cổng 443 (HTTPS) vào container.
- -p 8080:80: Ánh xạ cổng 80 vào cổng 8080 trên máy cần.
- haproxy: image của Docker (mặc định là phiên bản mới nhất).

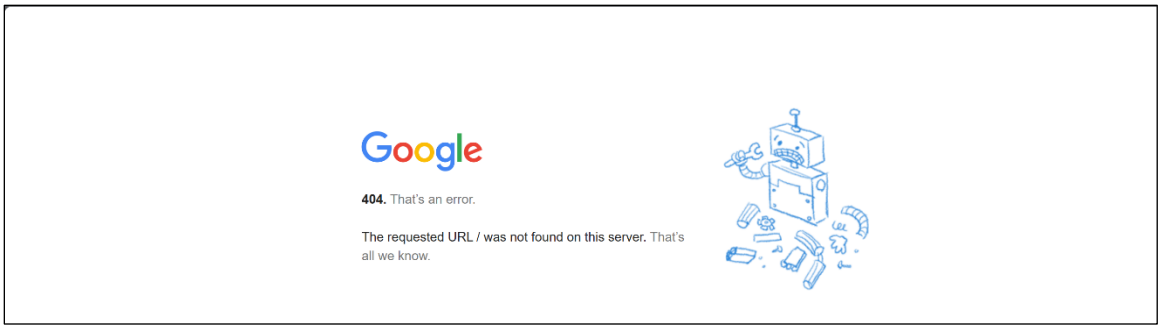
- Kiểm tra kết quả

- Truy cập vào testhaproxy1.com:443 => (Proxy trỏ vào Webserver).

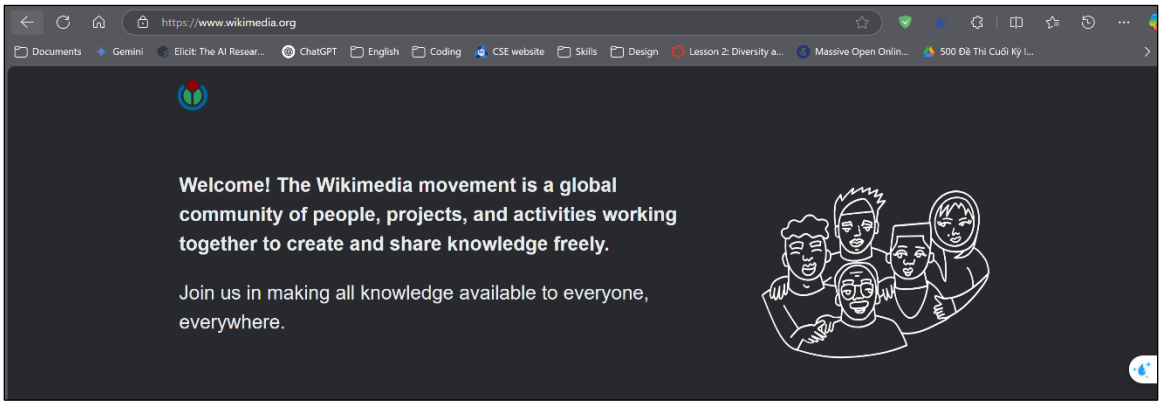


Hình 29: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy1.com:443 lần thứ nhất

- Truy cập vào testhaproxy1.com:443 (lần thứ 2) => Webserver phân phối theo giải thuật round-robin

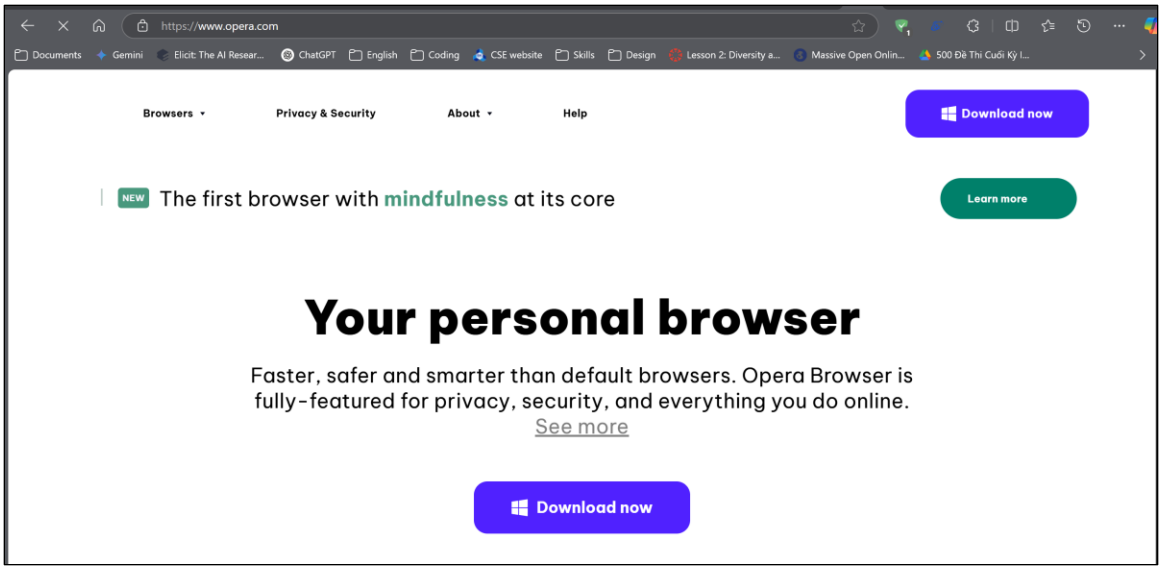


Hình 30: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy1.com lần thứ hai



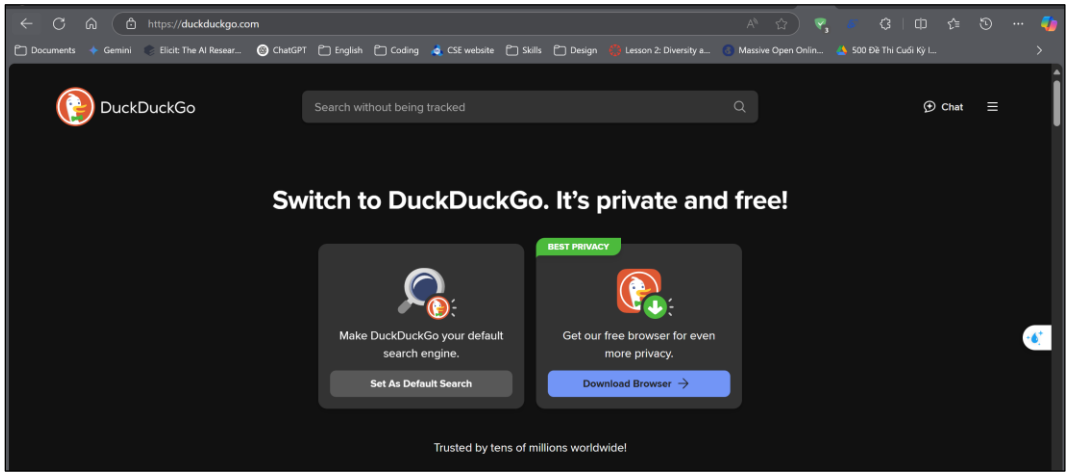
Hình 31: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy1.com:443 lần thứ ba

- Truy cập testhaproxy2.com:443 => Proxy frontend sau khi kiểm tra trong ACL sẽ phân phối tới Webserver1.



Hình 32: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy2.com:443

- Truy cập testhaproxy3.com:443 => Proxy frontend sau khi kiểm tra trong ACL sẽ phân phối tới Webserver2.



Hình 33: Client gửi yêu cầu kết nối tới Proxy Server qua testhaproxy3.com:443

Bảng 2: Bảng phân chia công việc

NHÓM 16

STT	HỌ VÀ TÊN THÀNH VIÊN	CHỨC VỤ	CÔNG VIỆC
1	Nguyễn Lê Nhật Đăng	Trưởng nhóm	- Lên kế hoạch họp nhóm, phân công công việc. - Tham gia nghiên cứu đồ án (SSL/TLS, ACL) - Viết nội dung và tổng duyệt báo cáo.
2	Trần Hải Đăng	Thành viên	- Chuẩn bị slide thuyết trình, video demo đồ án. - Viết nội dung báo cáo. - Tham gia nghiên cứu đồ án (Cơ sở lý thuyết).
3	Huỳnh Minh Đạt	Thành viên	- Tham gia nghiên cứu đồ án (Layer 4, Layer 7). - Viết nội dung và tổng duyệt báo cáo, slide thuyết trình. - Chuẩn bị nội dung, kịch bản thuyết trình.

Bảng 3: Bảng tự đánh giá thành viên

NHÓM 16

Tiêu chí / Thành viên	Thời gian (Mục 1)	Hoàn thành công việc (Mục 2)	Tham gia đóng góp ý kiến (Mục 3)	Chất lượng công việc (Mục 4)	Tinh thần hợp tác trong công việc (Mục 5)
Nguyễn Lê Nhật Đăng	Tốt	Tốt	Tốt	Tốt	Tốt
Trần Hải Đăng	Tốt	Tốt	Tốt	Tốt	Tốt
Huỳnh Minh Đạt	Tốt	Tốt	Tốt	Tốt	Tốt

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày …… tháng …… năm 2025

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)